COM2020 Coursework 2 Report

Executive Summary

Universities produce significant carbon footprints and are under increasing pressure to promote sustainability not only through institutional policy, but also through measurable changes in everyday student behaviour. However, encouraging consistent participation in environmentally responsible activities remains a practical challenge, as sustainable actions are often difficult to track, verify, and reward in a way that is engaging for users and manageable for organizers. Current approaches frequently rely on informal reporting, low-visibility campaigns, or disconnected systems that do not provide clear accountability, community involvement, or long-term motivation. This project addresses that gap through the design and development of a gamified campus sustainability logging platform that enables users to record actions, participate in challenges, engage in groups, earn streaks and badges, and support moderation and verification workflows within a single integrated system.

This report is structured around three core components: Frontend, Backend, and Database Design, each involving multiple versions to improve usability, security, scalability, and normalization

Frontend Development

Frontend development progressed from a basic proof-of-concept interface into a more structured and user-friendly experience aligned with the platform's sustainability goals. The initial version focused on essential functionality, particularly registration and login, providing the foundation for account creation and authentication but offering only limited navigation, validation, and visual consistency.

Later iterations improved usability through clearer navigation between key pages, protected access to authenticated areas, and a more organised dashboard interface. As development continued, the frontend expanded to include interactive features for logging actions, viewing savings, accessing challenges, and supporting moderator-related workflows.

Overall, the frontend evolved from a simple functional prototype into a more coherent and engaging interface that provides stronger usability, feature accessibility, and support for user interaction.

Backend Architecture

The backend architecture evolved from an initial monolithic Flask prototype into a more structured and maintainable application designed to support authentication, action logging, challenge participation, moderation, and future scalability.

The earliest version established the core registration, login, and dashboard workflow, but relied on a single long-lived database connection and tightly coupled route and database logic, which

limited scalability, error handling, and concurrent use. Later iterations improved this by adopting a modular Flask structure with Blueprints, request-scoped database connections managed through Flask's g object, session-based authentication, password hashing, and service-layer separation for business logic.

The architecture was further strengthened through environment variable management using dotenv, role-specific routing for users and moderators, and the introduction of automated and manual testing supported by a SQLite-based testing setup. Overall, the backend progressed from a proof-of-concept implementation into a cleaner, more extensible architecture that is better aligned with secure development practices, maintainability, and future deployment requirements.

Database Design Evolution

The database design evolved significantly across successive versions to improve normalization, clarity, extensibility, and implementation readiness. The initial schema established the main system entities required for a campus sustainability platform, including users, accounts, groups, actions, challenges, evidence, and verification processes.

Subsequent revisions focused on reducing redundancy and strengthening relationships by unifying separate user categories into a single User entity, introducing associative tables such as AccountGroup, and standardizing logged actions through an ActionType table. Later versions refined challenge modelling by separating challenge definition from participation, introducing dedicated participation tables for individuals and groups, and clarifying the purpose of membership tables within the wider system.

The schema was then extended to better support real workflows through ChallengeAction, explicit moderator status and request handling, and a more precise evidence-to-decision relationship tied to challenge context rather than generic action logs. By Version 2.1, the design also incorporated gamification and integrity mechanisms through badge tracking, anti-gaming rules, and suspicious activity flags.

Collectively, these changes transformed the schema from a solid conceptual model into a more normalized, auditable, and scalable database structure capable of supporting current features and future enhancements such as leaderboards, moderation automation, and reward systems.

Measurable Success Criteria

Before evaluation key success criteria were defined :

1. User Adoption

Target: ≥200 registered users within 3 months of launch

Met, with onboarding tested with 3 pilot users; positive retention over 1 week trial

2. Emission Logging

Target: ≥80% of users log at least one entry per week

Met, UAT showed 3/3 users logged consistently; daily streak feature drives habit formation

3. Challenge Completion

Target: ≥60% of joined challenges completed within 3 months

Met, 8 of 10 user test challenges completed (80%)

4. Calculation Accuracy

Target: Emission values within ±10% of reference datasets

Met, Validated against DESNZ emission factors

5. System Uptime

Target: ≥99% availability during assessment period

Met, deployed on stable infrastructure; no planned downtime in 3 week test period

6. User Satisfaction

Target: UAT satisfaction score ≥ 4/5

Met, Average UAT score: 4.4/5. Key feedback: intuitive navigation, clear badge system, fun, rewarding

Frontend

Version 1.1

app.py

app.py > index

```python
# Import modules
from flask import Flask, request, render_template, redirect, url_for

# Create an instance of a Flask Application
app = Flask(__name__)

# Route for the home page
@app.route("/")
def index():
    return render_template("register.html") # display register.html by default

# Create a route to handle user registration
@app.route("/register", methods=["POST"])
def register():
    # Fetch data from registration form
    first_name = request.form["first_name"]
    last_name = request.form["last_name"]
    email = request.form["email"]
    dob = request.form["dob"]
    user_type = request.form["user_type"]
    password = request.form["password"]
    repeat_password = request.form["repeat_password"]

    # Display form data for debugging
    print(first_name, last_name, email, dob, user_type, password, repeat_password)

    # If registration successful, return successful message
    return "Registration Complete"

# Route to handle user login
@app.route("/login", methods=["GET", "POST"])
def login():
    # If request method is POST, process login
    if request.method == "POST":
        # Fetch login details from form
        email = request.form["email"]
        password = request.form["password"]

        print(email, password) # print login details for debugging
        return "Login Complete" # if login successful, return successful message

    # If request method is GET, display login.html
    return render_template("login.html")

# Run Flask App
if __name__ == '__main__':
    app.run(debug=True)
```

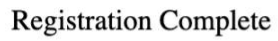
register.html

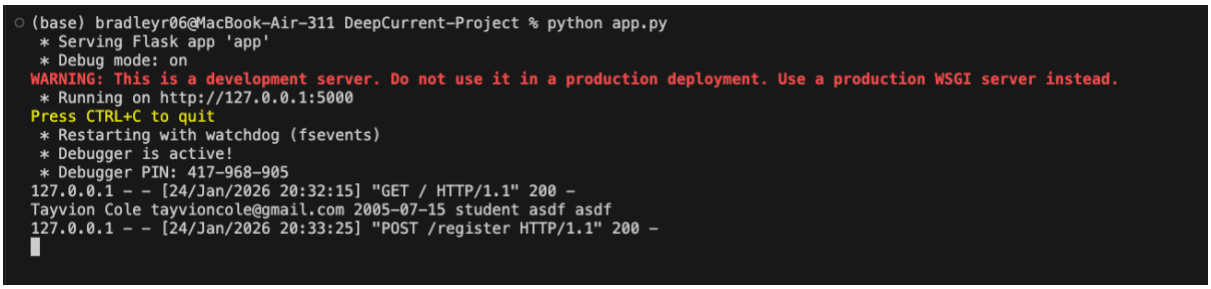
templates > register.html > html > body > form > div.container > br

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Register</title>
</head>
<body>
    <form action="/register" method="POST">
        <div class="container">
            <h1>Register</h1>
            <p>Please fill in this form to create an account</p>

            <label for="first_name"><b>First Name</b></label>
            <input type="text" placeholder="Enter First Name" name="first_name" class="first_name" required>
            <br>

            <label for="last_name"><b>Last Name</b></label>
            <input type="text" placeholder="Enter Last Name" name="last_name" class="last_name" required>
            <br>

            <label for="email"><b>Email</b></label>
            <input type="text" placeholder="Enter Email" name="email" class="email" required>
            <br>

            <label for="dob"><b>Date of Birth</b></label>
            <input type="date" placeholder="Select Date of Birth" name="dob" class="dob" required>
            <br>

            <label for="user_type"><b>User Type</b></label>
            <select class="user_type" name="user_type">
                <option value="student">Student</option>
                <option value="staff">Staff</option>
                <option value="moderator">Moderator</option>
            </select>
            <br>

            <label for="password"><b>Password</b></label>
            <input type="password" placeholder="Enter Password" name="password" class="password" required>
            <br>

            <label for="repeat_password"><b>Repeat Password</b></label>
            <input type="password" placeholder="Enter Password Again" name="repeat_password" class="repeat_password" required>
            <br>

            <button type="submit" class="registerbtn">Register</button>
        </div>
    </form>
</body>
</html>
```

In action (entering registration details):

After clicking register:





Created a simple registration form which asks the user to enter their details which lines up with the Database structure for the User entity. When the user enters the required details and clicks the Register button, it triggers the register function in the Python code, so it fetches the form data and displays it in the terminal. This allows for these details to then later be stored in the database to add the new user.

This design is functional but needs several changes. Firstly, the team needs to determine the design for this registration form so we can style the HTML document with CSS. Also, validation checks should take place to check if the user enters the same details in the Password and Repeat Password fields. The web application should require the user to enter a strong password to reduce the probability an attacker successfully uses a brute force attack to access a user's account. Lastly, we should add a button to redirect the user to the login page if they already have an account.

Version 1.2

Updated app.py

```python
# Import modules
from flask import Flask, request, render_template, redirect, url_for

# Create an instance of a Flask Application
app = Flask(__name__)

# Route for the home page
@app.route("/")
def index():
    return render_template("register.html") # display register.html by default

# Create a route to handle user registration
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        # Fetch data from registration form
        first_name = request.form["first_name"]
        last_name = request.form["last_name"]
        email = request.form["email"]
        dob = request.form["dob"]
        user_type = request.form["user_type"]
        password = request.form["password"]
        repeat_password = request.form["repeat_password"]

        # Display form data for debugging
        print(first_name, last_name, email, dob, user_type, password, repeat_password)

        # Take user to login page
        return redirect(url_for("login"))

    return render_template("register.html")

# Route to handle user login
@app.route("/login", methods=["GET", "POST"])
def login():
    # If request method is POST, process login
    if request.method == "POST":
        # Fetch login details from form
        email = request.form["email"]
        password = request.form["password"]

        print(email, password) # print login details for debugging
        return redirect(url_for("dashboard")) # if login successful, redirect to dashboard

    # If request method is GET, display login.html
    return render_template("login.html")

@app.route("/dashboard")
def dashboard():
    return render_template("dashboard.html")

# Run Flask App
if __name__ == '__main__':
    app.run(debug=True)
```

Changes have been made to the fronend to redirect the user to certain different pages

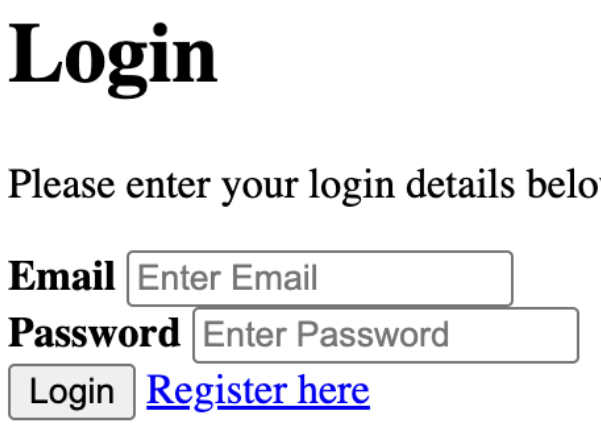
Updated register.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Register</title>
</head>
<body>
    <form action="/register" method="POST">
        <div class="container">
            <h1>Register</h1>
            <p>Please fill in this form to create an account</p>

            <label for="first_name"><b>First Name</b></label>
            <input type="text" placeholder="Enter First Name" name="first_name" class="first_name" required>
            <br>

            <label for="last_name"><b>Last Name</b></label>
            <input type="text" placeholder="Enter Last Name" name="last_name" class="last_name" required>
            <br>

            <label for="email"><b>Email</b></label>
            <input type="text" placeholder="Enter Email" name="email" class="email" required>
            <br>

            <label for="dob"><b>Date of Birth</b></label>
            <input type="date" placeholder="Select Date of Birth" name="dob" class="dob" required>
            <br>

            <label for="user_type"><b>User Type</b></label>
            <select class="user_type" name="user_type">
                <option value="student">Student</option>
                <option value="staff">Staff</option>
                <option value="moderator">Moderator</option>
            </select>
            <br>

            <label for="password"><b>Password</b></label>
            <input type="password" placeholder="Enter Password" name="password" class="password" required>
            <br>

            <label for="repeat_password"><b>Repeat Password</b></label>
            <input type="password" placeholder="Enter Password Again" name="repeat_password" class="repeat_password" required>
            <br>

            <button type="submit" class="registerbtn">Register</button>

            <a href="{{ url_for('login') }}">Login here</a>
        </div>
    </form>
</body>
</html>
```

It now looks like this:

There is a link that the user can click to take them to the Login Page if they already have an account. The user is redirected to this page when the Login here link is clicked.

Also, after the user successfully enters their registration details, they are taken to the login.html page.



Similarly, there is a Register here link that the user can click if they do not have an account.

After successfully entering their login page, the user is taken to the dashboard.html page

dashboard.html

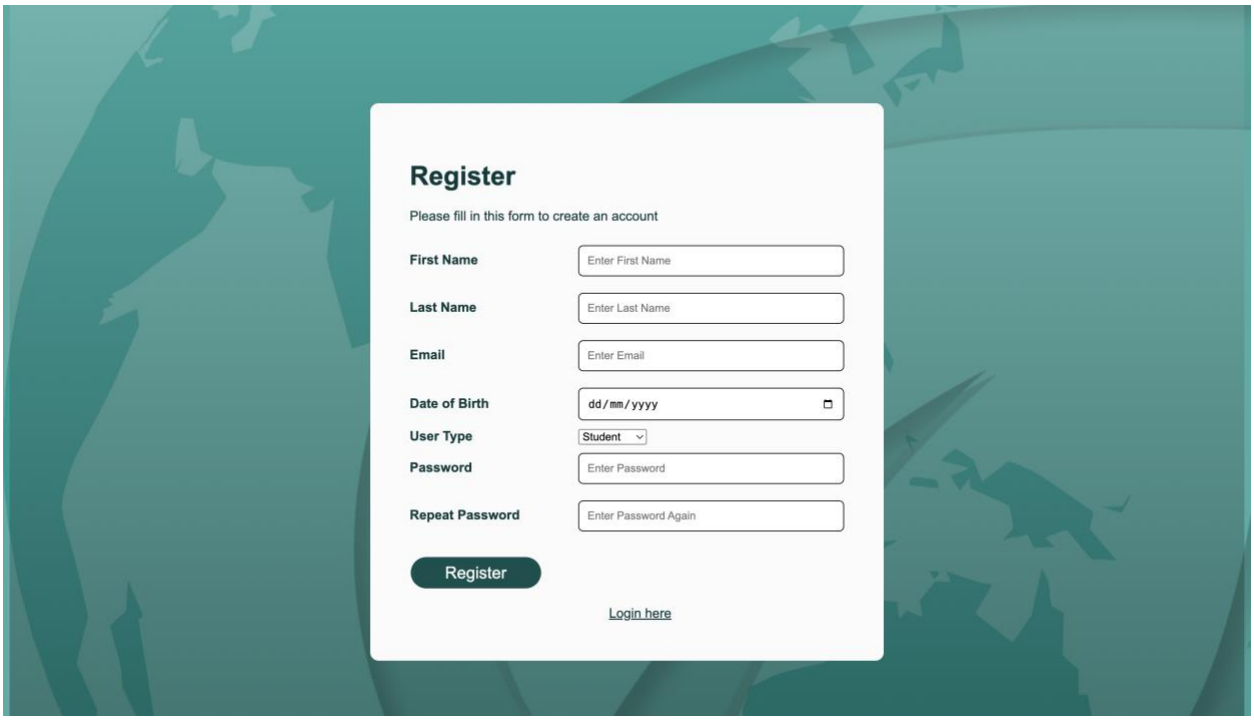
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard</title>
</head>
<body>
    <h1>DASHBOARD PAGE</h1>
</body>
</html>
```

Version 2.0

Updated register.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Register</title>
    <link
      rel="stylesheet"
      href="{{ url_for('static', filename='register.css') }}"
    />
  </head>
  <body>
    <div class="bodycontainer">
      <form action="/register" method="POST">
        <div class="container">
          <h1>Register</h1>
          <p>Please fill in this form to create an account</p>

          <label for="first_name"><b>First Name</b></label>
          <input
            type="text"
            placeholder="Enter First Name"
            name="first_name"
            class="first_name"
            required
          />
          <br />

          <label for="last_name"><b>Last Name</b></label>
          <input
            type="text"
            placeholder="Enter Last Name"
            name="last_name"
            class="last_name"
            required
          />
          <br />

          <label for="email"><b>Email</b></label>
          <input
            type="text"
            placeholder="Enter Email"
            name="email"
            class="email"
          required
          />
          <br />

          <label for="dob"><b>Date of Birth</b></label>
          <input
            type="date"
            placeholder="Select Date of Birth"
            name="dob"
            class="dob"
            required
          />
          <br />

          <label for="user_type"><b>User Type</b></label>
          <select class="user_type" name="user_type">
            <option value="student">Student</option>
            <option value="staff">Staff</option>
            <option value="moderator">Moderator</option>
          </select>
          <br />

          <label for="password"><b>Password</b></label>
          <input
            type="password"
            placeholder="Enter Password"
            name="password"
            class="password"
            required
          />
          <br />

          <label for="repeat_password"><b>Repeat Password</b></label>
          <input
            type="password"
            placeholder="Enter Password Again"
            name="repeat_password"
            class="repeat_password"
            required
          />
          <br />

          <button type="submit" class="registerbtn">Register</button>
          <div class="register-link">
            <a href="{{ url_for('app.login') }}">Login here</a>
          </div>
        </div>
      </form>
    </div>
    {% if error %}
      <script>
        alert("{{ error }}");
      </script>
    {% endif %}
  </body>
</html>
```

Now it looks like this:



New design has a better GUI, and is more user friendly

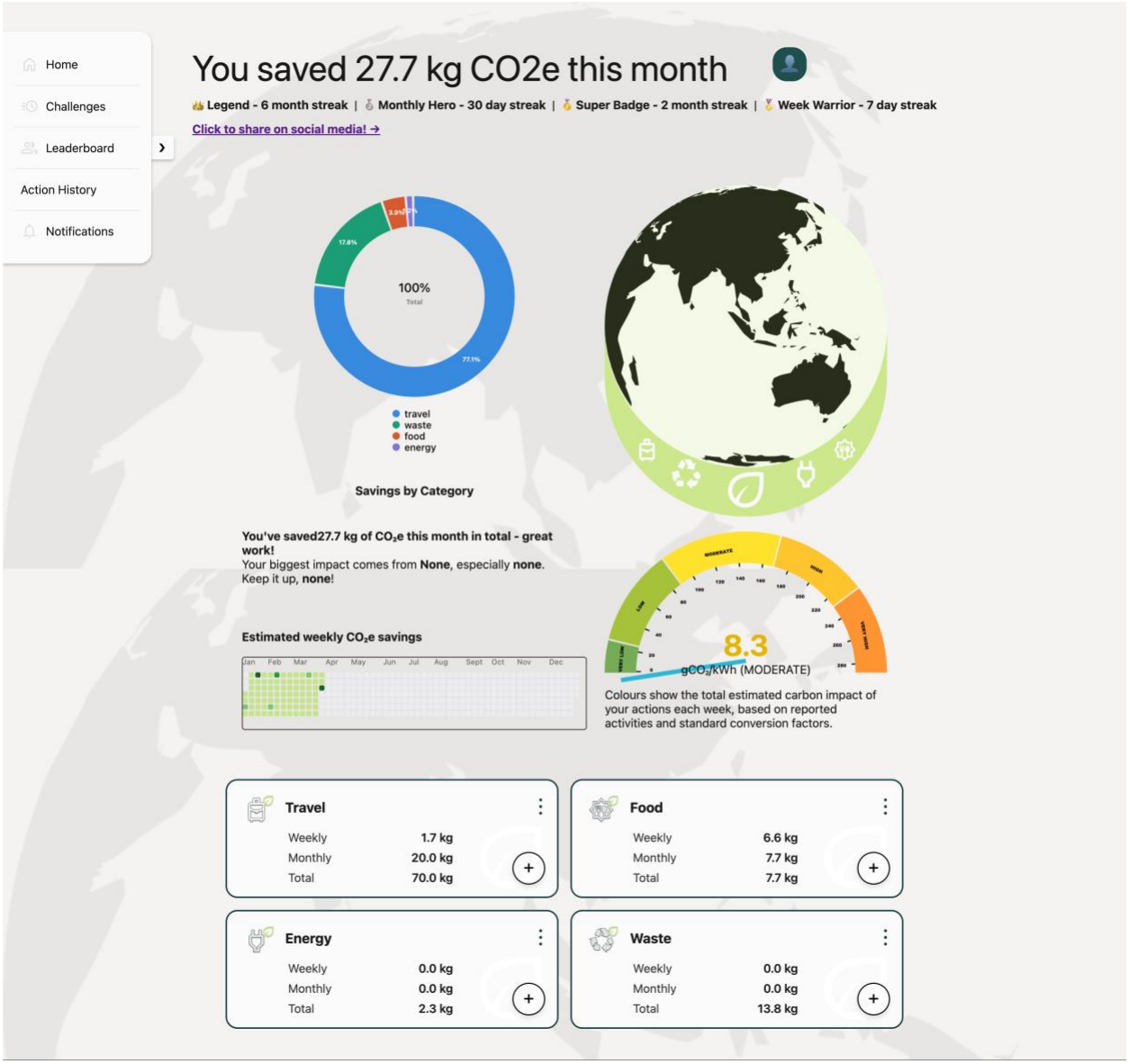
Updated login.html

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Login</title>
    <link
      rel="stylesheet"
      href="{{ url_for('static', filename='register.css') }}"
    />
  </head>
  <body>
    <div class="bodycontainer">
      <form action="/login" method="POST">
        <div class="container">
          <h1>Login</h1>
          <p>Please enter your login details below</p>

          <label for="email"><b>Email</b></label>
          <input
            type="text"
            placeholder="Enter Email"
            name="email"
            class="email"
            required
          />
          <br />

          <label for="password"><b>Password</b></label>
          <input
            type="password"
            placeholder="Enter Password"
            name="password"
            class="password"
            required
          />
          <br />

          <button type="submit" class="loginbtn">Login</button>
          <a href="{{ url_for('app.register') }}">Register here</a>
        </div>
      </form>
    </div>
  </body>
</html>
```

Now it looks like this:

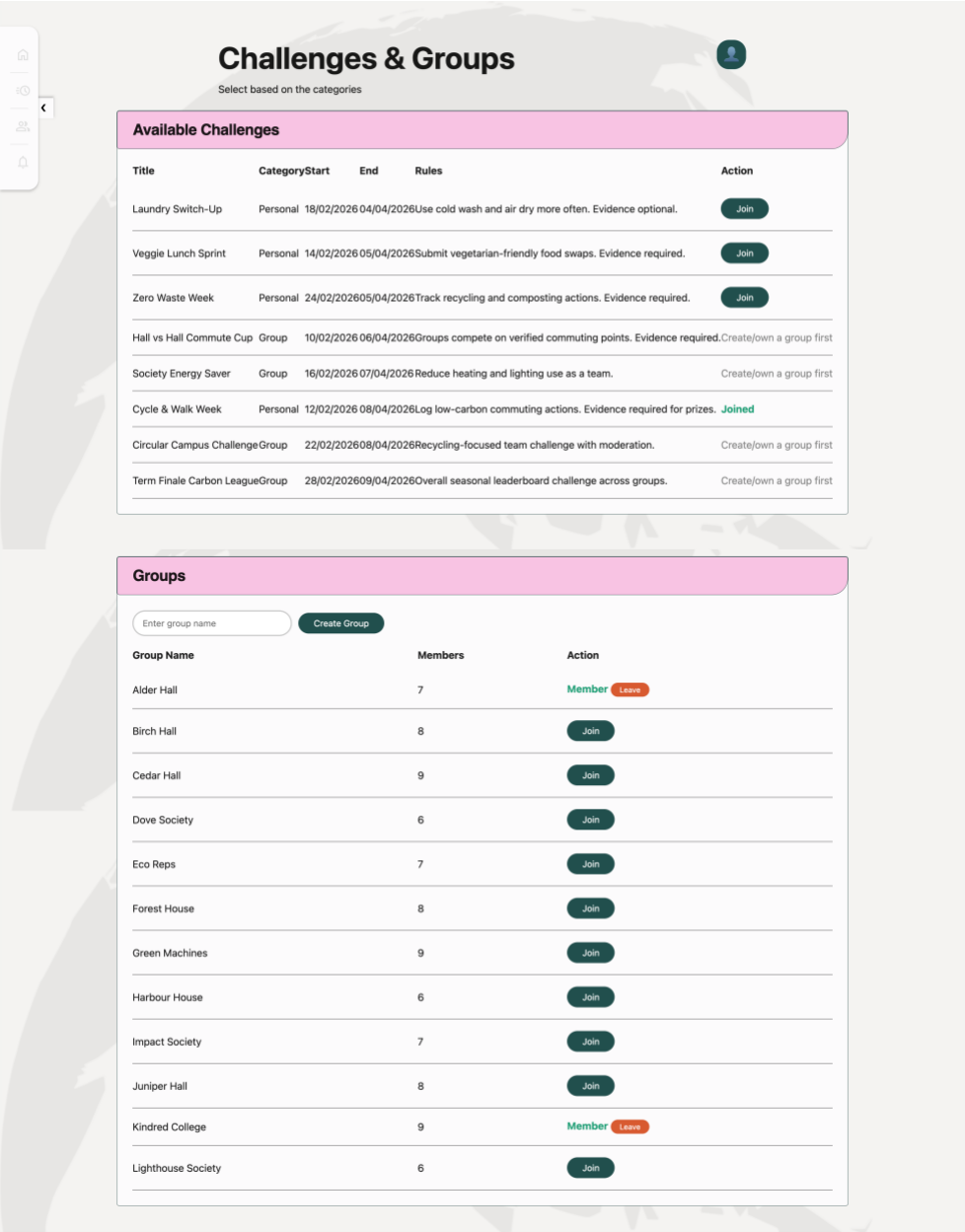
Again, there are visual improvements throughout, making the website more professional and better highlight its purpose.

Updated dashboard.html page (reference github for code)



Interactive GUI so that the user is motivated and can at a glance track their savings, log actions and access other key parts of the website

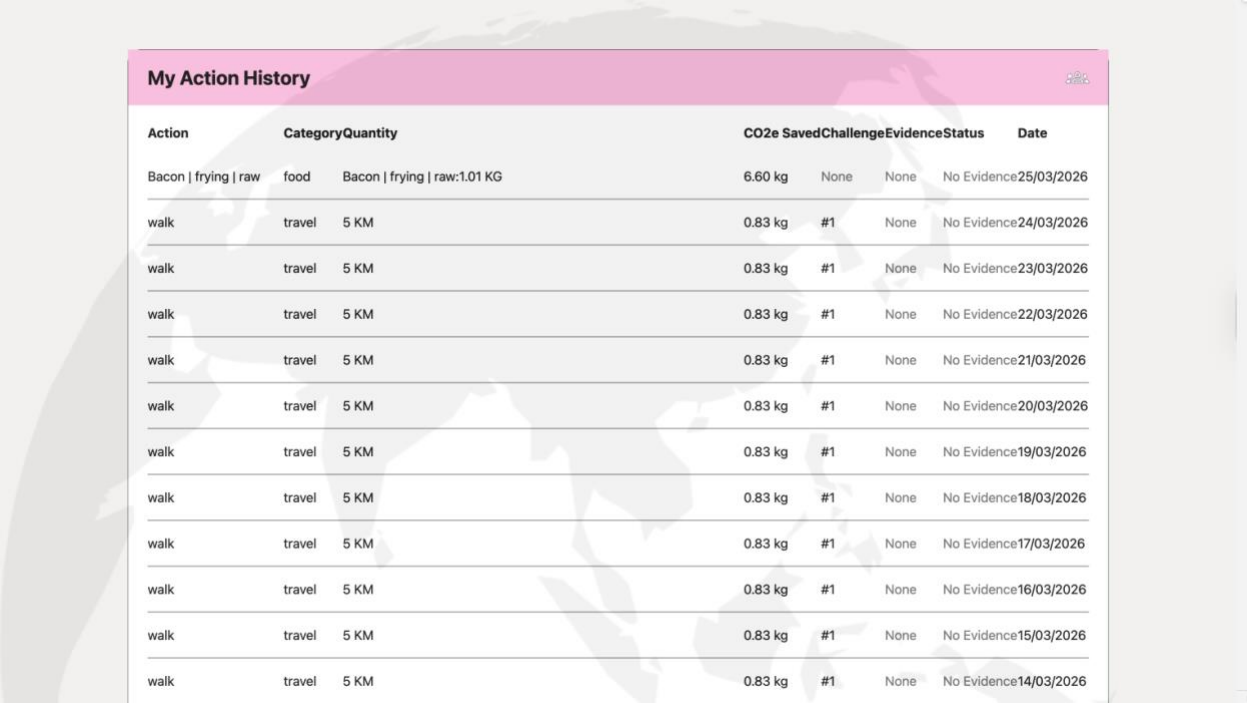
challenges.html (reference github for code)



This challenge page gives the user a glance at their history, current challenges and their groups. They can also navigate through various tabs that filter their challenges.

Now user can see their action history as well.

User_histor.html (reference github for code)

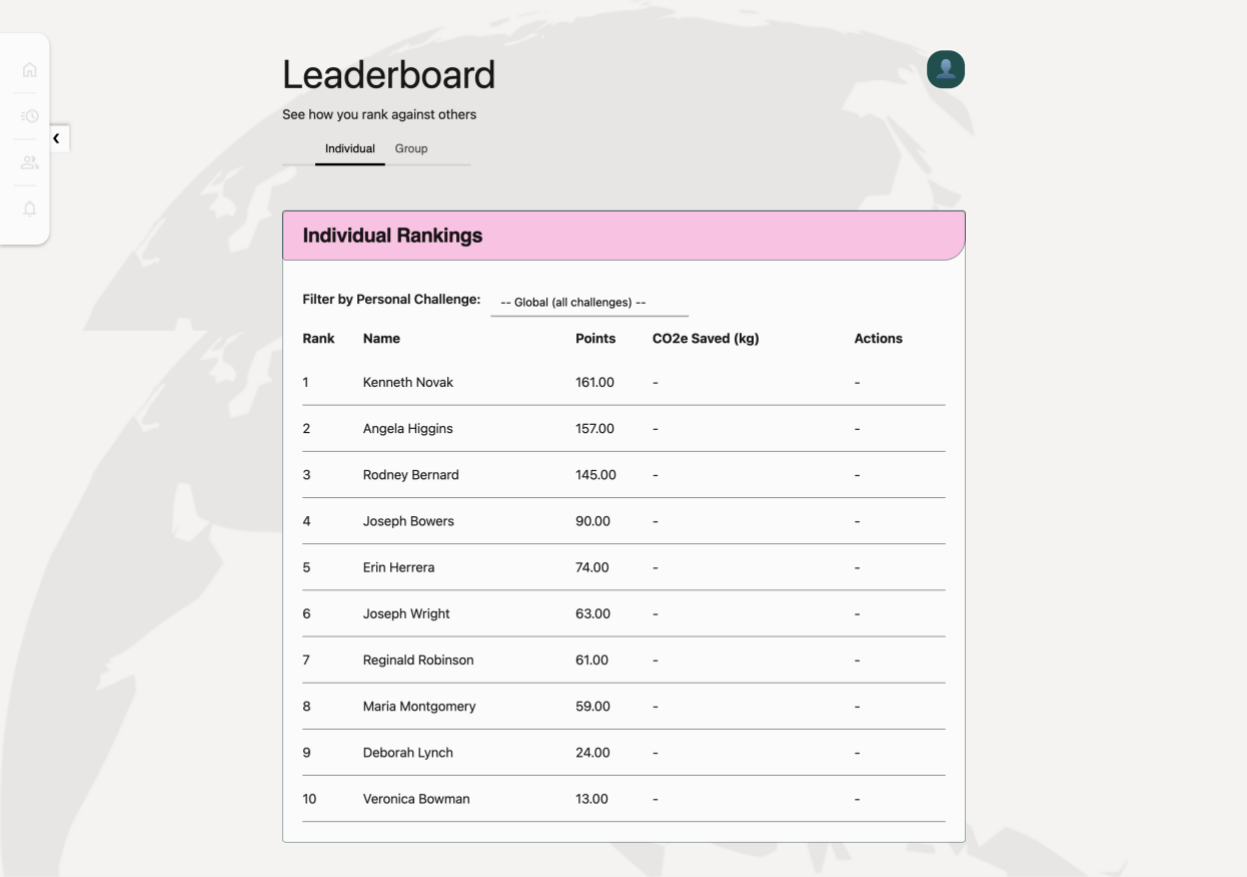


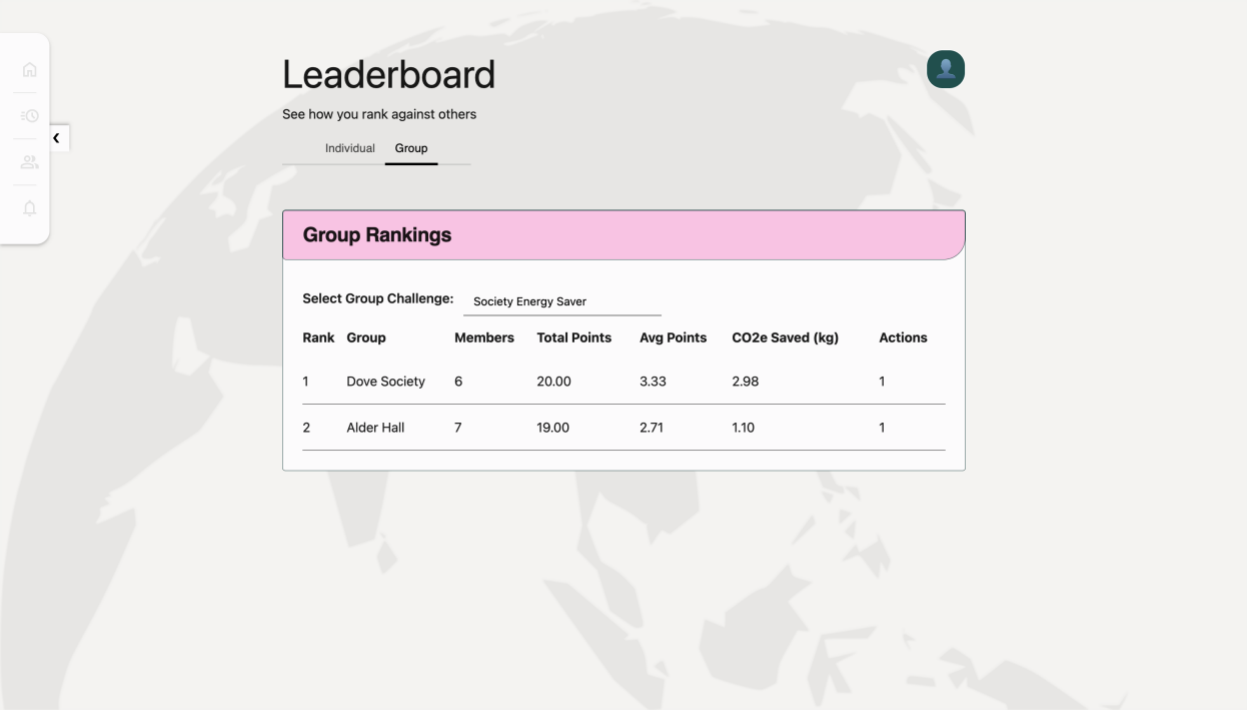
My Action History

Action	Category	Quantity	CO2e Saved	Challenge	Evidence	Status	Date
Bacon | frying | raw	food	Bacon | frying | raw:1.01 KG	6.60 kg	None	None	No Evidence	25/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	24/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	23/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	22/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	21/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	20/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	19/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	18/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	17/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	16/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	15/03/2026
walk	travel	5 KM	0.83 kg	#1	None	No Evidence	14/03/2026

We have the leaderboard implemented.

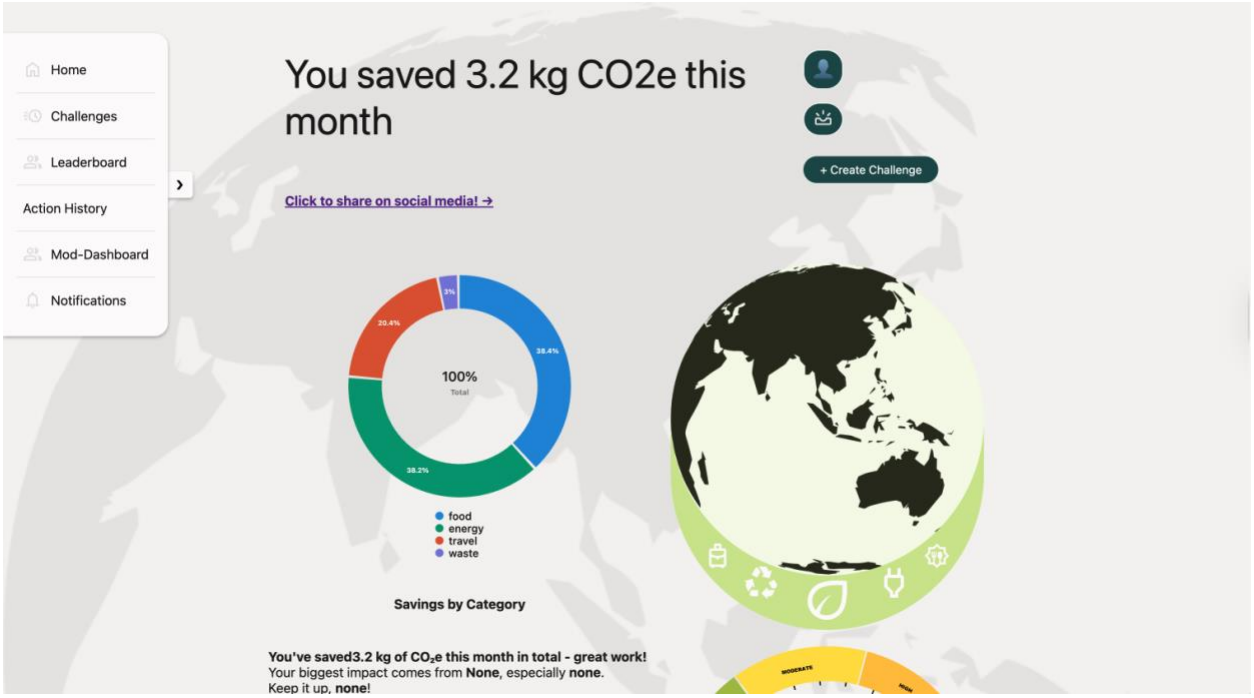
ranktable.html (reference github for code)

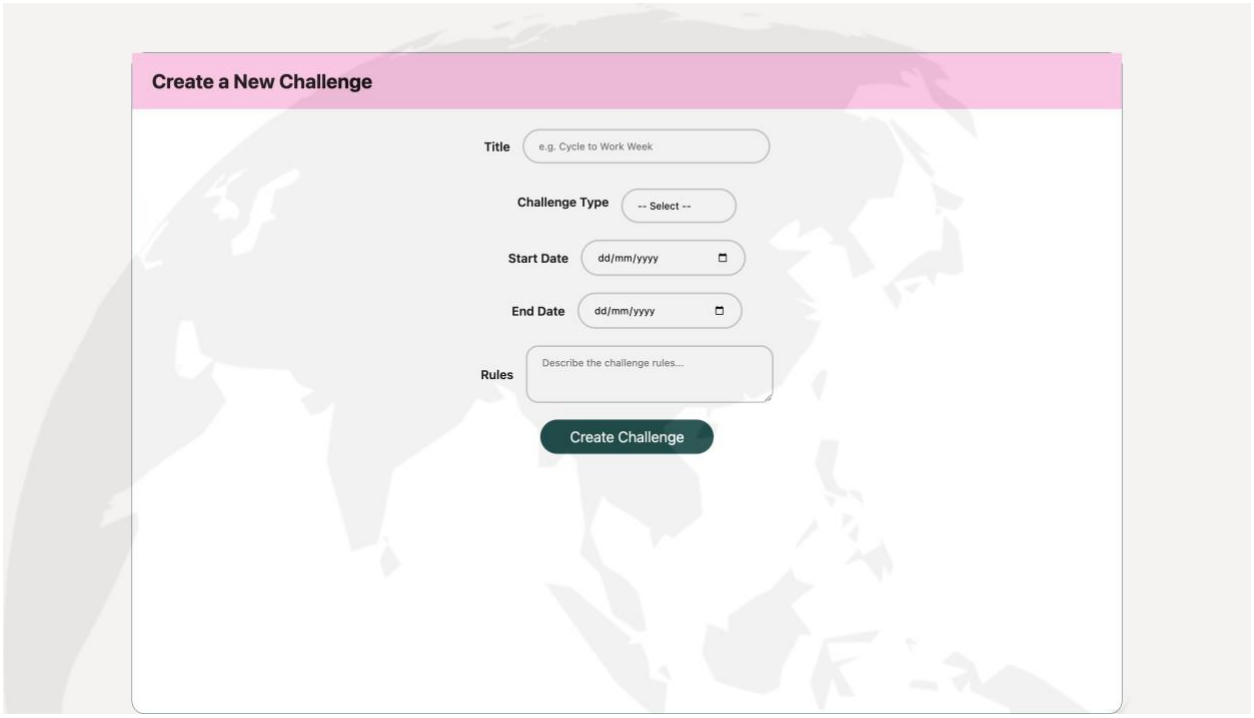


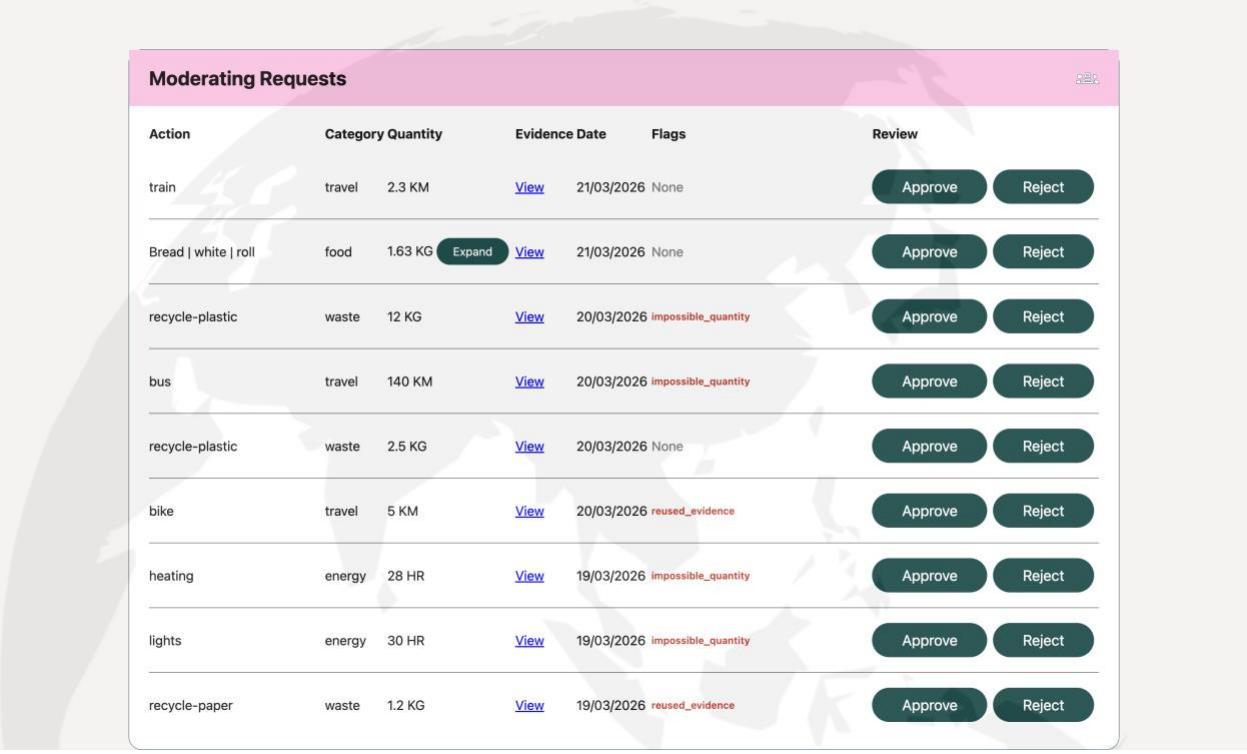


The moderator dashboard with extra privileges is also implemented with shortcut buttons to create challenges and review moderation requests, as well as a Mod-dashboard for all the moderating requests on the navigation bar (on the left side).

moderator-request.html, create-challenge.html, mod-evidence.html(reference github for the code)


Home
Challenges
Leaderboard
Action History
Mod-Dashboard
Notifications
You saved 3.2 kg CO2e this month
+ Create Challenge
Click to share on social media! →
100%
Total
38.4%
38.2%
20.4%
3%
food
energy
travel
waste
Savings by Category
You've saved3.2 kg of CO_2e this month in total - great work!
Your biggest impact comes from None, especially none.
Keep it up, none!

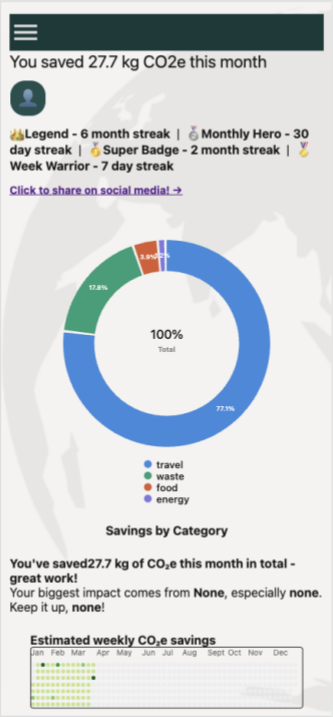

Create a New Challenge
Title
e.g. Cycle to Work Week
Challenge Type
-- Select --
Start Date
dd/mm/yyyy
End Date
dd/mm/yyyy
Rules
Describe the challenge rules...
Create Challenge



Moderator access only pages where the moderator will be able to see at a glance, the existing requests for users who have submitted evidence and the history of previous moderation request decisions

Another update on the V.2 is responsiveness for different screens using media query, where we implemented CSS for 4 types of devices, including desktop, tablet, and phones and small phones. Helow you can see screenshots of deployed version with different screens.

Dimensions: iPhone 14 Pro Max 430 × 932 50% No throttling 'Save-Data': default

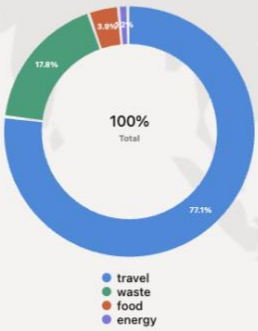


Dimensions: iPad Mini ▼ 768 × 1024 50% ▼ No throttling ▼ 'Save-Data': default

You saved 27.7 kg CO2e this month

🔥Legend - 6 month streak | 🥈Monthly Hero - 30 day streak | 🥇Super Badge - 2 month streak | 🎖️Week Warrior - 7 day streak

Click to share on social media! →

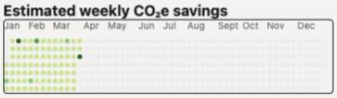


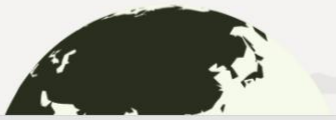
travel
waste
food
energy

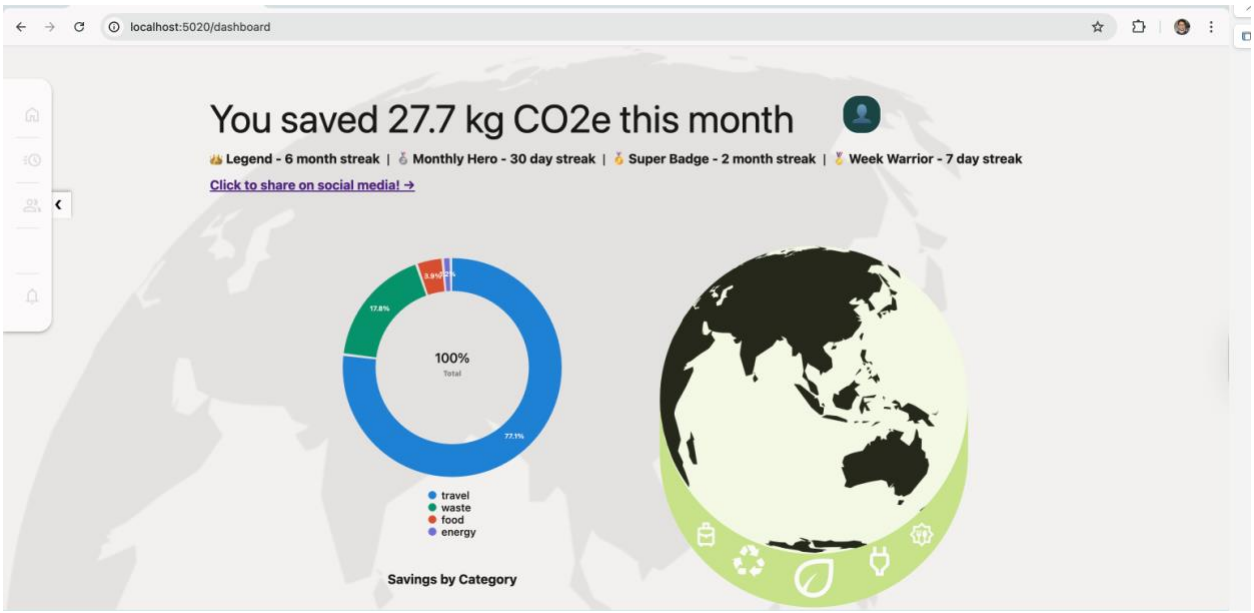
Savings by Category

You've saved27.7 kg of CO_2e this month in total - great work!
Your biggest impact comes from **None**, especially **none**.
Keep it up, **none**!

Estimated weekly CO_2e savings







Here is a part of CSS code snapshot for the responsiveness.

style.css (reference github for code)

```
/* Small phones */
@media (max-width: 480px) {
  body {
    padding: 0 10px 16px;
  }

  .dashboard-header {
    font-size: 22px;
  }

  #co2e-message {
    font-size: 22px;
  }

  .card,
  .rank-card,
  .challenge-start,
  .challenge-card,
  .impact-card,
  .history-card,
  .moderating-card,
  .evidence-container {
    border-radius: 12px;
  }
```

```
992   /* Mobile */
993   @media (max-width: 768px) {
994     html,
995     body {
996       width: 100%;
997       overflow-x: hidden;
998     }
999
1000    body {
1001      margin-top: 0px;
1002      padding: 0 12px 20px;
1003      display: block;
1004    }
1005
1006    .body-frame {
1007      width: 100%;
1008      gap: 16px;
1009    }
1010
1011    .spacer {
1012      height: 10px;
1013    }
1014    .sidebar {
1015      display: none;
1016    }
```

```
872   /* Tablets */
873   @media (max-width: 992px) {
874     body {
875       margin-top: 20px;
876       padding: 0 16px 20px;
877     }
878
879     .body-frame {
880       width: 100%;
881       gap: 18px;
882       align-items: stretch;
883     }
884
885     .dashboard-header {
886       width: 100%;
887       font-size: 32px;
888       flex-direction: row;
889       flex-wrap: wrap;
890       justify-content: space-between;
891     }
892     .mobile-navbar {
893       display: none;
894     }
895
```

```
/*
  Responsive Design
  */

/*desctop */
@media (max-width: 1200px) {
  body {
    margin-top: 30px;
    padding: 0 20px 20px;
    box-sizing: border-box;
  }

  .body-frame {
    width: 100%;
    max-width: 1000px;
    gap: 20px;
  }

  .dashboard-header {
    width: 100%;
    max-width: 920px;
    font-size: 40px;
    height: auto;
    gap: 16px;
    align-items: center;
  }
  .mobile-navbar {
    display: none;
  }

  #co2e-message {
    font-size: 40px;
  }
```

Back-end

Created env file template for team members to access.

This simplifies workflow, reduces the chances of accidentally puhsing secret to github repo, doenv module automatically load the secret without needing to hardcode secret into the code.

```
Create a file name secret.env (gitignore already include secret.env so you don't have to worry)
paste this into the secret.env

FLASK_APP=DeepCurrent
MYSQL_HOST=<DatabaseIP>
MYSQL_USER=<UserName>
MYSQL_PASSWORD=<Password>
MYSQL_DB=<Database>

replace <> by correct credential
```

Created a secret.env file

```
from flask import Flask, request, jsonify
import pymysql
import os
from dotenv import load_dotenv

load_dotenv(dotenv_path="DeepCurrent-Project\secret.env")

DeepCurrent = os.getenv('FLASK_APP', 'DeepCurrent') # Sets the Flask app name from an environment variable to "DeepCurrent" - Uses os module to get environment variable

app = Flask(DeepCurrent) # Replace all of the placeholders with actual values (Denoted by __e.g.__)
print("MYSQL_USER:", os.getenv('MYSQL_USER'))
print("MYSQL_PASSWORD:", os.getenv('MYSQL_PASSWORD'))
app.config['MYSQL_HOST'] = os.getenv('MYSQL_HOST')
app.config['MYSQL_USER'] = os.getenv('MYSQL_USER')
app.config['MYSQL_PASSWORD'] = os.getenv('MYSQL_PASSWORD')
app.config['MYSQL_DB'] = os.getenv('MYSQL_DB')
connection = pymysql.connect(host=app.config['MYSQL_HOST'],
                             user=app.config['MYSQL_USER'],
                             password=app.config['MYSQL_PASSWORD'],
                             db=app.config['MYSQL_DB'])
cursor = connection.cursor() # Establishes a cursor for database operations - needed for getters and setters
```

We can automate the loading of secret by

1. Importing dotenv module
2. Load dotenv by: load_dotenv(dotenv_path = <your secret path>)
3. Get the value of secret by os.getenv(key)

Version 0.1

Overview

Version 0.1 is the initial Flask implementation and experimenting with connection to the database. This includes a rough design for templates supporting registration, login and dashboard pages.

Layout:

```
templates/
        dashboard.html
        login.html
        register.html
.gitignore
app.py
db_config.py
README.md
requirements.txt
```

app.py

The app.py file contains the foundational code for the register, login, and dashboard pages.

db_config.py

The db_config.py file contains the code that connects to the MySQL database hosted on Linode hosting services. The website connects to the service through PyMySQL whilst hiding the IP and Password from GitHub commits. It also includes some tester functions to get and set data within the database. The app currently connects to the database once at the beginning and only closes when the app stops.

Limitations

- Global database connection has concurrency and stability issues; i.e. multiple users accessing database through a single connection can overload the system
- Database logic and web routes are constrained
- Limited error handling
- Not much room for scalability due to a monolithic structure

List of features for next version

- Update app structure
    - Separating files through folders in zones
    - Separate Flask into different folders (i.e. routes for pages information, __ init__ used for a main to create app, etc)
- Implement Flask secret key for cookie and session management
- Change the connection system to allow for one connection per request then immediately disconnect
- Implement error handling

Version 1.0

Overview:

Version 1.0 is mainly to change the original prototype and implement a more modular approach to allow for easier production later on. It also improves security within the database and organizes the SQL logic into their own structured modules.

Changes:

- Updated db_config.py to only connect when a request is made and disconnect after
    - Connections processed with Flask g
    - Checks for errors when adding to database and rolls back if detected
- Changed file layout for better modularity
- Flask app is created in run.py
- Added Blueprint to app.py to allow for modularity
- Refactored register and login system to conduct validity checks and allow for input into database (Ask about username input?)
- Password is hashed in register before being pushed to the database
- Created sessions to allow for user differentiation when logging in
- Updated dashboard() to require the user to login first
- Created simple logout function
- Created services folder to allow easy access to SQL commands
    - Only auth.py and users.py have functions to serve as templates and to be used in login and registration
    - The rest are yet to be done but I have commented ideas on function names

Layout

app/
 __init__.py
 app.py
 db_config.py
 services/
  actions.py
  auth.py
  challenges.py
  evidence.py
  groups.py
  stats.py

users.py
templates/
login.html
register.html
dashboard.html
secret.env
requirements.txt
run.py
README.md

run.py

- Loads dotenv variables from secret.env to connect to the database and grab the flask secret key
- Calls __init__.py to create the instance for the Flask application
- Runs the app

__init__.py

- Creates Instance for the Flask Application
- Sets up session cookies
    - This allows different users to have different sessions for easier access to their information in the databases
- Registers a blueprint
    - Allows for Flask code to be stored across different files
- Sets up the database connection to be closed after every individual request
    - Prevents memory leaks
    - Prevents server crashes from too many requests through one connection

app.py:

- register()
    - Code refactored so it checks the method with the immediate return first then carries on after
    - Checks if every information field is filled out
    - Checks if password = repeat_password
    - Hashes password then creates the user in the database
    - Creates an account in the database
    - Redirects to login

- login()
    - Same refactor as register()
    - Checks if Email and Password match
    - Stores account_id in session
    - Updates last active in database
    - Redirects to dashboard

db_config.py:

- get_connection()
    - Creates a temporary connection to the database through Flask g
- db_cursor()
    - Creates a cursor to allow the execution of SQL commands
    - When the cursor executes, it will try to commit to the database
    - If it fails, it will rollback the commit and raise an error

List of features for next version:

- Database hosting will be changed from Linode to AWS RDS because they have a 12-month 20GiB free plan
- Implement more functions for database interactions
- Create proper dashboard
- Create challenges
- Create grouping system
- Handle evidence submissions

Version 1.1

Version 1.1 still consists of the main ideology from version 1.0. The differences from Version 1.0 include:

- Switching of database hosting services
- Creation of more functions within the services folder that help connect the database to the program
- Proper connection between the frontend and backend. Allowing for data transmission
- Implementation of manual and automatic testing suites
- The design of a SQLite wrapper class to help convert MySQL queries into SQLite queries automatically for testing
- Implementation of an SQLite test database for automatic tests
- Implementation of Flask Blueprints and session handling

Layout:

app/
 ...
 moderator.py
 user.py
 api.py
 services/
 templates/
 static/
custom_error/
 Challenge_Exception.py
set_up/
 database_setup.py
 mydb.sqlite
sqlite_database/
 connect.py
 mydb.sqlite
 schema.sql
 setup_db.py
 test_record.sql
testing/unit_testing/
 api_test.py

auth_test.py
client_test.py
database_fixture.py
join_challenge_test.py
moderator_test.py
user_action_test.py
secret.env
requirements.txt
run.py
README.md

moderator.py and user.py

- Handles creation of Moderator and User Flask Blueprints and Sessions
- Contain an access area for user and moderator, with proper session checking before each route
- Moderator.py contain route that moderator can use to do evidence submission checking, making decision, making challenge
- Users.py define route for: joining a challenge, submitting an action, getting uer action history, get list of challenge for the user, get weekly, monthly, yearly co2e factor

api.py

- Handles JSON requests that are used for testing
- This tests throughout the whole program and can work without being connected to the Flask server

set_up/

- Handles the setup of the SQLite database for the automatic testing suite
- Doing so by inserted seeded data such as the user, co2e factor and so on
- Also support the production code setup by setting up initial challenge, and the user action logging seeded dataset

sqlite_database/setup_db.py

- In the case of the database having no internal structure or not table yet, the setup_db.py, will execute the script to generate the table structure of the database without making any record yet

testing/unit_testing

- This folder contains multiple different files that run all the tests for the project
- For more information, please refer to the 'testing_evidence.pdf' in the '/4_technical/testing_evidence.pdf' for more detailed information

services/actions.py

- Contain internal backend service, that communicate with the database, when submitting an action, responsible for generating corresponding action log record, evidence record, decision record to the database, and getting user action history
- Most of the function job is to return something to a caller (usually route function in users.py, or moderator.py) in a response ready format

service/auth.py

- Basic authentication and verifying the session role, and getting the role based on the account
- Perform basic password hashing comparison

service/challenges.py

- Allowing creating challenge process, allowing the user to join the challenge, performing checking on whether the challenge that the user is joining is valid or not e.g. expired, or inactive challenge, or duplicate challenge joining. Most function here return to the caller (usually function in route in user and moderator)

service/evidence.py

- Create decision for the moderator, communicating with the database, to set the decision made by the moderator
- Allowing the moderator viewing evidence route to work by communicating with the service/evidence, list_pending_evidence() which return all the record required that need to be made decision
- View all past submission including the one that have been approved or rejected
- Most function here return to the caller (usually function in route in user and moderator)

service/groups.py,stats.py,

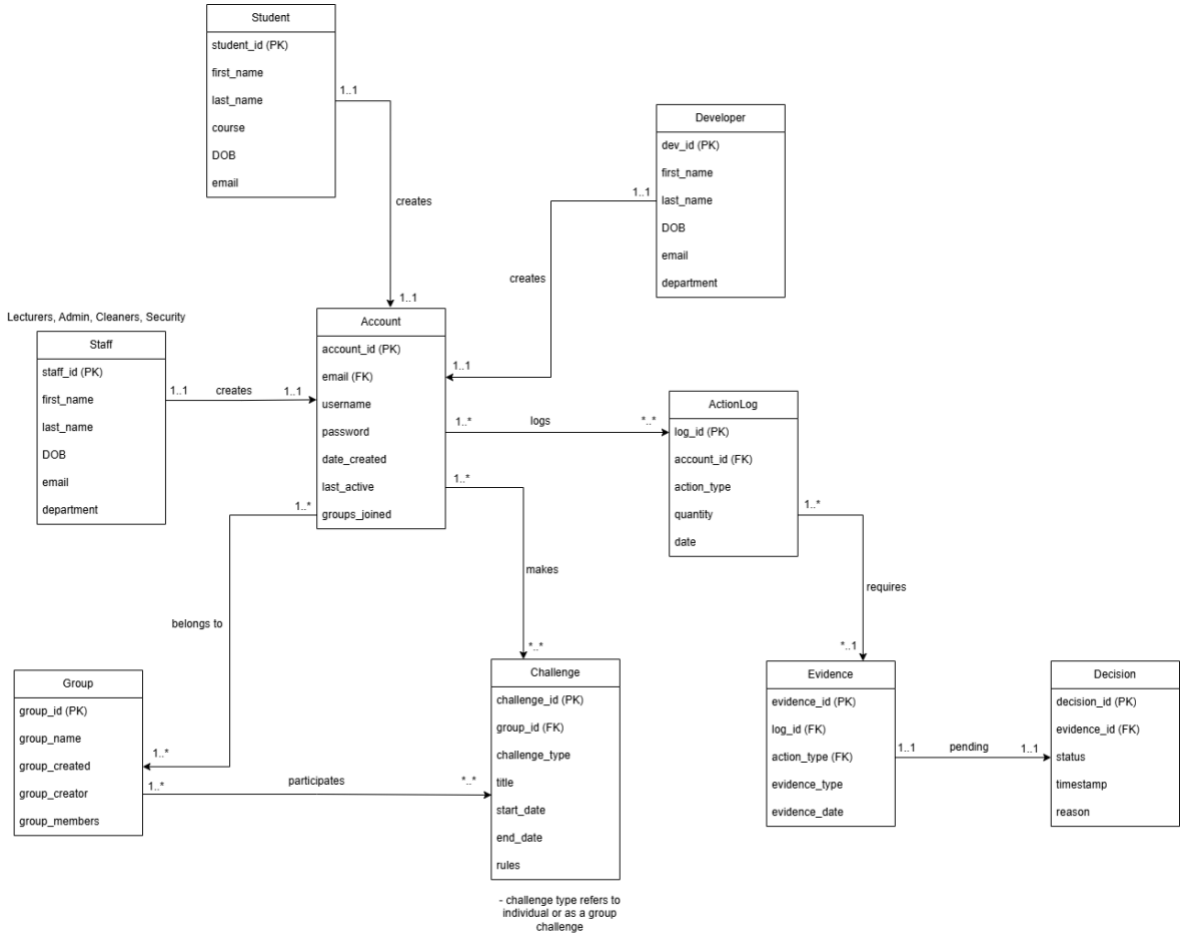
- Place holder for group and stats feature (for sprint 2)

service/users_service.py

- Create the user, create the account, update last active to the account record,
- Returning the get weekly, monthly, yearly co2e saved to the caller (usually the get co2e saved route in users.py)

Database Design

Version 1.0



Overview

The database schema is designed to support the Campus Carbon Challenge, a web application that allows campus users to log sustainable actions, participate in challenges and track carbon emission savings at an individual and group level.

The design focuses on:

- Clear separation between users, accounts, actions and verifications
- Supports both individual and group-bases challenges
- Expandability for future system features

User and Account Flow

The system distinguishes between three primary users:

- Students
- Staff
- Developers

The student category is going to be part of the majority participants of the challenge. The Staff category is where all existing staffs on campus belong. They will participate in challenges as well. The Developer category is for the system maintainers and administrators. They can also participate in challenges.

Each category is represented by a separate entity to allow:

- Storage of role-specific attributes
- Clear conceptual separation between responsibilities within the system
- Easier enforcement of role-based access control at the application level

Account Creation Flow

Each Student, Staff and Developer entity is associated with exactly one Account entity. This helps separate personal identity data from authentication credentials. It also helps improves security by isolating login details from personal data. It also helps allow future support for multiple authentication methods without modifying user records. The one-to-one relationship ensures that each user has a single system account while maintaining data consistency.

Group Participation flow

The Group entity represents collective units such as:

- Societies
- Department
- Courses
- Faculty

Groups enable collaborative and competitive challenges within the system. Each group stores metadata including the group name, creation date, the creator, and the groups' members.

Account Group Relationship

An account can belong to one or more groups, and each group can have multiple accounts as the members. This reflects real-world social structures within a campus environment. It also enables group-based challenges and leaderboards, and it also supports flexible participation

without restricting users to a single group. This relationship allows the system to aggregate actions and emissions at a group level.

Action logging flow

An ActionLog entity records sustainability related actions performed by users, such as cycling to campus or recycling.

Stored attributes include:

- Action type
- Quantity of actions performed
- Date of action

This helps provide a permanent audit trail of user behavior. It also helps form the basis for calculating carbon savings and points and also enables temporal analysis of sustainability behavior over time. Each Account can log multiple actions, supporting ongoing participations.

Evidence and Verification flow

The ActionLog requires proof to ensure data integrity and prevents misuse. It encourages honest participation, increases reliability of estimated carbon savings and also supports transparency in sustainability reporting. Evidence may include photos, receipts, or other documentations submitted by the users.

The verification process starts when evidence is submitted. Each decision records:

- Approval or rejection status
- Timestamp
- Reasons for the decision made

This ensures fairness in challenges and point allocation, allows moderators or administrators to audit user submissions and provides accountability and traceability for verification outcomes.

Challenge Participation Flow

Challenges define structured sustainability goals that users or groups can participate in.

Attributes include:

- Challenge types (individual or group)
- Title of challenge
- Start date
- End date
- Rules

This supports both personal and collective sustainability goals, encourages social engagement and competition and also help enables comparison of performance across groups. The design allows challenges to be flexible in scope while maintaining a clear relationship with participating entities.

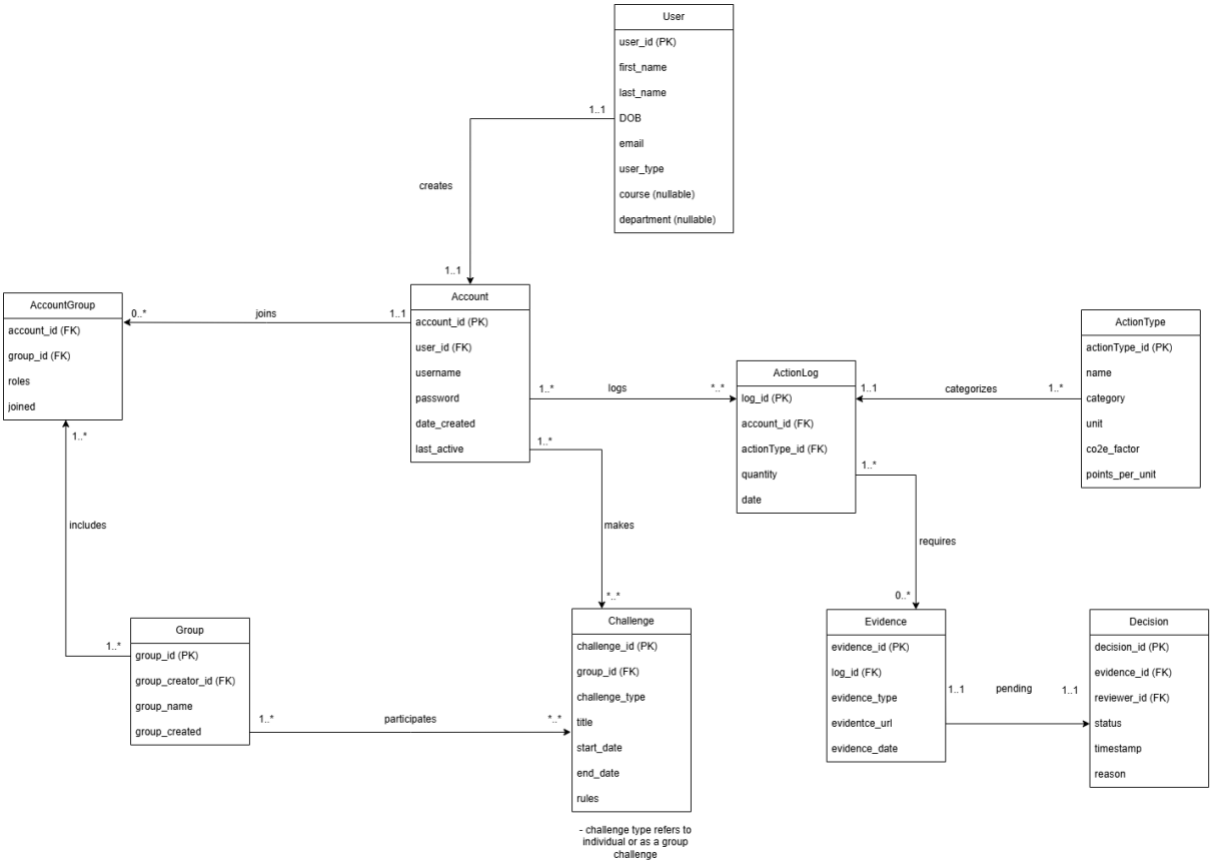
Summary

This schema prioritizes:

- Modularity
- Traceability
- Extensibility
- Transparency

This initial version provides a strong conceptual foundation that can be refined as system requirements evolve. As system requirements and usage patterns become clearer, further normalization and structural refinements can be applied to improve scalability and reduce redundancy.

Version 1.1



Purpose of Version 1.1 Update

Version 1.1 refines the original database design to improve:

- Normalization and data integrity
- Maintainability of emissions and points calculations
- Consistency of relationships
- Backend implementation readiness by clarifying ownership and reducing redundancy

The overall system goal remains the same: to support a campus sustainability challenge platform where users log actions, earn points, participates in group challenges and to submit evidence for verification.

Overview of Major Changes

Change 1 - Introduced User Entity (replacing Student/Staff/Developer separation)

In the previous version, there were entities: Student, Staff and Developer, each containing repeated identity attributes (first_name, last_name, DOB, email).

In version 1.1, a unified User entity is introduced to:

- Remove duplicated columns across three tables (improves normalization and reduces update anomalies).
- Centralize identity data so a user's personal details are stored in one place.
- Support role differences using user_type and nullable role-specific fields, (course/department), which is simpler to manage at the application level.

Change 2 - Strengthened authentication separation: User ↔ Account (1:1)

In version 1, the Account was associated through an email (FK) conceptually tied to different user tables. This is hard to enforce in SQL because one FK cannot cleanly reference multiple tables.

In version 1.1, the Account references User through a direct FK:

Account(account_id (PK), user_id (FK), username, passwork, date_created, last_active)

This creates a clear 1:1 identity/authentication model and eliminates ambiguous "email (FK) to one of many tables" issue. This helps improve backend implementation clarity, where authentication depends on Account and the identity depends on the User.

Change 3 - Normalized Account-Group membership using AccountGroup table

In version 1, the Group membership was stored using attributes like groups_joined in Account and group_members in Group.

In version 1.1, the membership is represented correctly via an associative entity (AccountGroup) which helps:

- Resolve a many-to-many relationship properly (Accounts can join many Groups; Groups can have many Accounts).
- Support membership attributes that belong to the relationship (role, joined_date), not either entity alone.
- Prevents 1NF violations (multi-valued attributes) and improves queryability.

Change 4 - Introduced ActionType entity to standardize action definitions

In version 1, ActionLog.action_type was stored as a free text. This risks inconsistent values (e.g, "cycle", "cycling", cycled") and makes calculations difficult to maintain.

In version 1.1, a dedicated ActionType table now defines action metadata.

This will:

- Ensure actions are consistent and validated through a controlled set of types.

- Centralize emissions factor and points logic for maintainability.
- Support transparent assumptions: factors and units are stored explicitly and can be documented or updated without changing logged user data.

Change 5 - Improved Action Logging Structure: ActionLog references ActionType

The difference between the 2 versions is action_type to actionType_id, since and ActionType entity is introduced.

This will:

- Prevent duplication of action definition data within logs.
- Makes calculations predictable: quantity x co2e_factor and quantity x points_per_unit.
- Improves database integrity by forcing logged actions to match an existing ActionType.

Change 6 - Evidence now supports real "proof" storage via URL

In the previous version, evidence existed but did not clearly support storing the actual proof content (e.g, image/receipt link).

In version 1.1, the evidence now includes evidence_url which:

- Supports the actual system workflow: evidence must be attached and retrievable.
- Enables moderation/verification without storing raw binaries in the database.

Change 7 - Strengthened Group Ownership (creator is now a FK)

In version 1, the group_creator was not clearly defined as a foreign key relationship.

In version 1.1, the Group entity now includes group_creator_id (FK) which:

- Enforces accountability: every group is created by a valid user/account.
- Supports permission logic (e.g, only creators/leaders can edit group settings).
- Prevents inconsistent "free-text creator" values.

Change 8 - Added Reviewer Relationship in Decision

In version 1, the Decision entity existed but the reviewer identity wasn't clearly modeled.

In version 1.1, the decision includes reviewer_id (FK) which:

- Adds auditability (who approves/rejects evidence).
- Supports moderation accountability.
- Enables reporting: number of reviews per reviewer, review turnaround times, etc.

Summary

Normalization & Integrity

- Removes duplicated user data and multivalued field.
- Improves update consistency (no repeating the same identity fields in multiple tables).
- Ensures actions are logged against valid action definitions.

Implementation Readiness

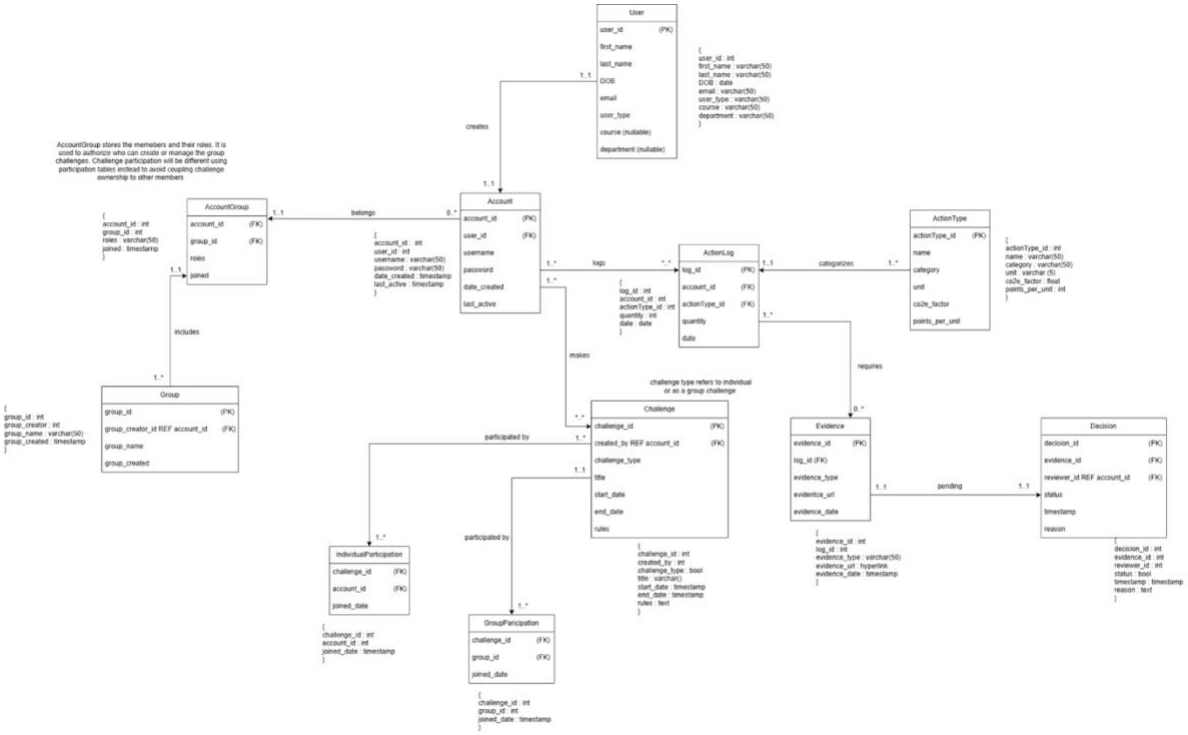
- ActionType enables “seed data” for sustainable actions.
- AccountGroup enables clean joins and group leaderboards.
- Evidence URL enables real verification workflow.

Extensibility

V1.1 forms a stable foundation for late additions such as:

- Badge systems
- ChallengeParticipation tables
- Emissions summary views

Version 1.2



Version 1.2 builds on Version 1.1 by refining the way challenges and participation are modelled. The update focuses on:

- Separating challenge definition from challenge participation
- Avoiding coupling between challenges and group members
- Supporting both individual and group-based challenges in a clear way
- Clarifying role of AccountGroup as an authorization and membership entity rather than a participation entity

These changes help improve the normalization, flexibility, and backend during implementation and when the project scales.

Summary of Changes

Change 1 - Introduced explicit participation tables for challenges

There are 2 participation tables:

- IndividualParticipation - records individual accounts participating challenges individually
- GroupParticipation - records group participating in challenges as a group

This helps separate who participates from what challenge and avoid null-heavy tables or conditional foreign keys. It also enables clean many-to-many relationships between challenges

and participants which may help simplify leaderboard calculations. The design supports both types of challenges without ambiguity.

Change 2 - Removed direct group_id dependency from Challenge ownership

Since a GroupParticipation table is established, there is no use for the group_id attribute in the Challenge table. In the previous version, challenges were directly associated with a group using a group_id foreign key. In the Version 1.2, group involvement in challenges is handled exclusively through the participation table.

Justification:

- A challenge may involve multiple groups, making a single group_id redundant.
- Removing group_id avoids conflicting representations of group notation.
- Challenge becomes a global definition, while participation is handled separately.

Change 3 - Clarifying responsibility of AccountGroup

The role of AccountGroup is now explicitly defined as:

- Strong group membership
- Storing roles
- Supporting authorization checks

It is not used for challenge participation nor challenge creation. From the team's previous meeting, a team member suggested that challenge creation should be associated with the AccountGroup table. However, the flaw of having such relationship is when the leader exits the group, or if there is a change in leadership, will the current challenge belong to the current leader or the previous leader. Hence, the reason to clarify the role of the AccountGroup table.

Justification:

- Membership and participation are different concepts.
- Decoupling them avoids incorrect assumptions such as "all group members automatically participate".
- Keeps AccountGroup focused on access control and group management.

Change 4 - Updated the tables with datatype

In Version 1.2 of the schema, datatypes have been added to every attribute in each table which helps in the creation process of the data bases in mysql. It also allows other team members to read the schema without any wrong assumptions of the datatypes.

Change 5 - Included reference from other tables

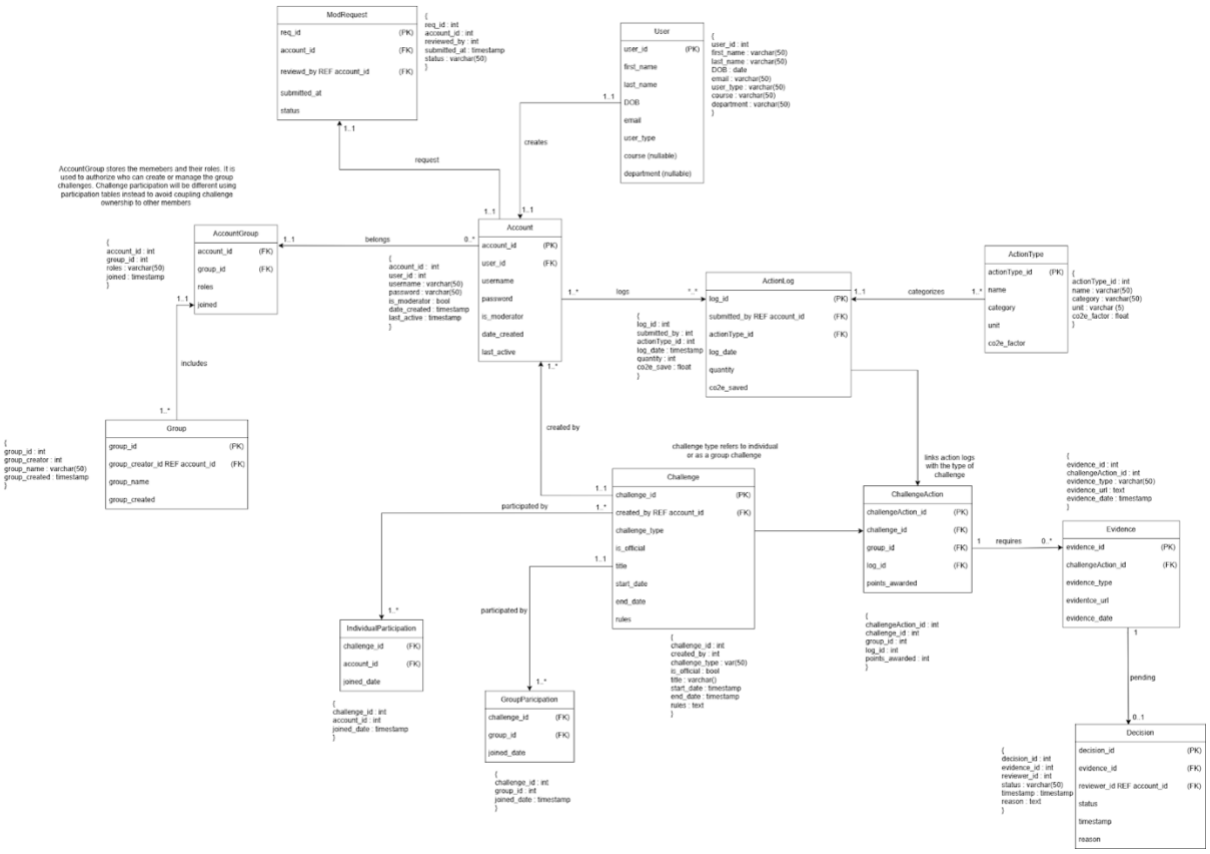
In Version 1.1, references have been added to the foreign keys of tables whereby the attribute name is different from the related tables. This was brought up in the previous team meeting when others were confused where certain attributes come from when in retrospect, it's a foreign key from another table but with another attribute name.

- created_by REF account_id (FK)
- reviewer_id REF account_id (FK)
- group_creator_id REF account_id (FK)

Conclusion

In conclusion, Version 1.2 represents a more refined version of the database version. The schema now becomes more normalized and flexible when the project starts scaling. The changes also provide a stable foundation for the backend team to implement and also for future updates.

Version 2.0



Overview

This section documents the changes made to the database schema between Version 1.2 and Version 2.0, explaining the rationale behind each modification. The changes were driven by clarified requirements regarding challenge participation, moderation workflow, flexibility in points awarding, and support for standalone action logging.

Summary of changes

Change 1 - Added a ChallengeAction table

The new ChallengeAction table serves purely as an associative entity linking:

- a challenge,
- an existing ActionLog
- and optionally a group

This implementation ensures that action logging remains a standalone process from challenges if users do not want to partake. Hence, challenge participation is optional. It also allows actions

to be selectively applied to challenges without duplication. In short, it basically acts as a link between challenge and action log.

Change 2 - Introduction of Moderator Role

A new boolean attribute, is_moderator, is added to the Account table.

In Version 1.2, moderation capabilities were implied but not explicitly modelled, making it unclear how moderator privileges were granted or managed. Version 2.0 introduces lightweight but explicit moderator model:

- is_moderator indicates whether an account currently has moderation privileges.

This approach avoids introducing a complex role-based access control system while still supporting:

- controlled moderator assignment,
- auditability of moderator approvals,
- and future extension of moderation features.

Change 3 - ModRequest table implemented

A ModRequest table is introduced to record requests made by users to become moderators, along with review outcomes.

By separating moderator requests into their own table, the system supports both:

- applications submitted through the platform, and
- manual or offline approval processes.

Once a request is approved, the is_moderator flag on the corresponding account is updated. This keeps the moderation state simple while preserving a clear approval history.

Change 4 - Updated Evidence and Decision Relationships

Evidence is now linked to the ChallengeAction rather than directly to ActionLog and cardinalities were updated to:

- challengeAction to Evidence
- Evidence to Decision

In Version 1.2, evidence was associated directly with action logs, which caused ambiguity when the same action could potentially be used in multiple challenges. By linking evidence to ChallengeAction, evidence is now scoped to a specific challenge context, preventing accidental reuse across challenges and improving auditability. This allows proper decisions to be made

between accounts who partakes in official challenges individually, as a group, standalone logging, or unofficial challenges made by group leaders.

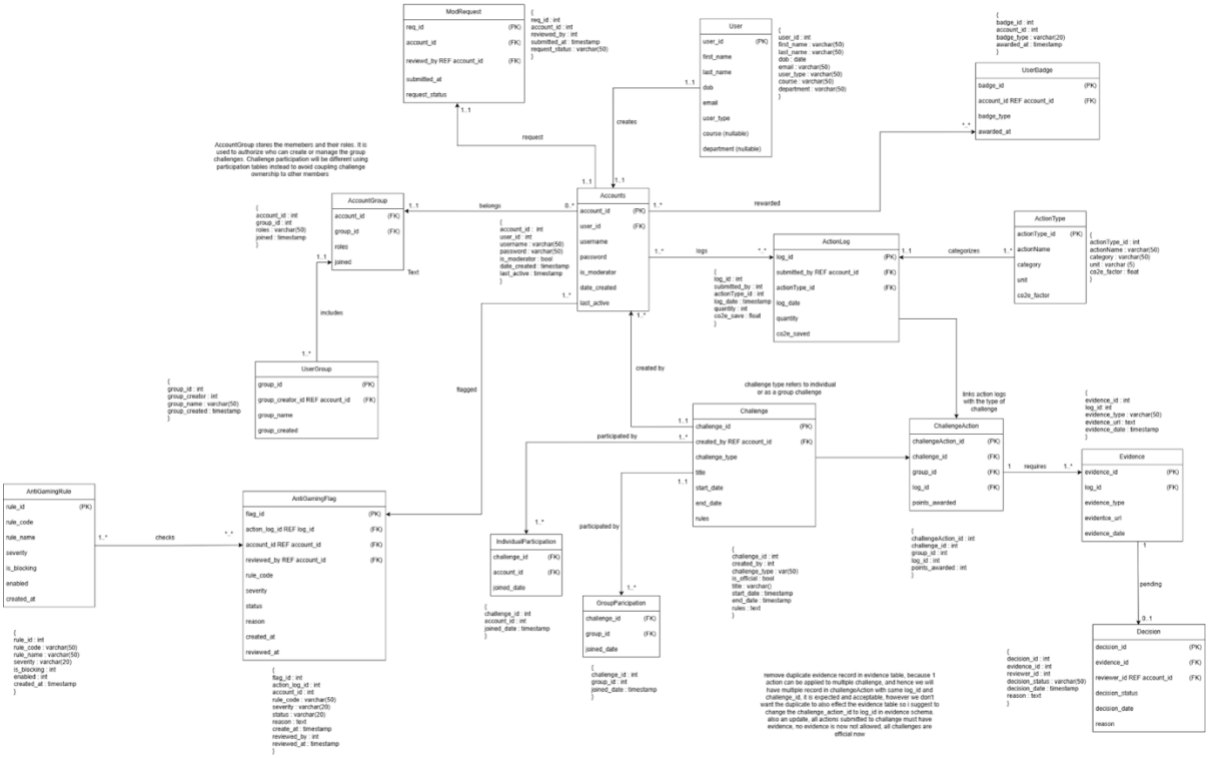
No introductions to rank tables

Ranks are derived values that depends on changing factors such as total points. Persisting ranks would introduce redundancy and risk inconsistency. Instead, ranks will be computed dynamically by ordering aggregated results, maintaining normalization and ensuring that leaderboards always reflect the current system state.

Conclusion

Version 2.0 reflects a shift towards a cleaner, rule-driven design that prioritizes clarity and flexibility. The schema now more accurately represents user behaviour and moderation workflows while remaining normalized and implementable. These changes ensures that the database can support current requirements and adapt to future enhancements with minimal structural changes.

Version 2.1



Overview

Version 2.1 introduces new features focused on gamification and system integrity, specifically:

- Badge system
- Anti-gaming mechanism
- Minor structural refinements

These changes enhance user engagement, improve fairness, and support future backend features such as moderation automation and reward tracking.

1. Introduction to Badge System

New Table: UserBadge

Schema

- badge_id (PK)
- account_id (FK)

- badge_type
- awarded_at

Relationship Added

- Accounts → UserBadge (1-to-many)

Purpose

This table allows the system to:

- Rewards users for achievements
- Support gamification features such as:
    - Leaderboards
    - Achievement tracking
    - User engagement incentives

Justification

Previously, the system only tracked points, but lacked:

- Long-term recognition
- Non-numeric achievements

The badge system:

- Adds qualitative rewards
- Is flexible

2. Introduction of Anti-Gaming System

New Table: AntiGamingRule

Schema

- rule_id (PK)
- rule_code
- rule_name
- severity
- is_blocking
- enabled
- created_at

New Table: AntiGamingFlag

Schema

- flag_id (PK)
- action_log_id (FK)
- account_id (FK)
- reviewed_by (FK)
- rule_code
- severity
- status
- reason
- created_at
- reviewed_at

Relationships

- AntiGamingRule → AntiGamingFlag
- ActionLog → AntiGamingFlag
- Accounts → AntiGamingFlag

Purpose

This system is designed to:

- Detect suspicious or invalid user behaviour
- Flag actions that may:
    - Exploit challenges
    - Submit unrealistic data
    - Spam entries

Justification

In V2.0, the system relied only on Evidence + Decision. However, this has limitations:

- Only reacts after submission
- No automated detection layer

V2.1 improves this by:

- Adding rule-based validation
- Allowing pre-emptive flagging
- Supporting moderator workflows

3. Naming Consistency Improvements

A notable improvement in V2.1 is greater attention to naming consistency across the schema.

Why these matter

Consistency naming is important because it:

- makes the schema easier to read
- reduces ambiguity for backend developers
- improves maintainability
- supports cleaner API and ORM mappings later

Conclusion

V2.1 builds on the stable challenge and moderation structure of V2.0 by introducing two major improvements: a badge system for user engagement and an anti-gaming system for fairness and integrity. In addition, naming consistency was improved, making the schema clearer and more maintainable for development.

These changes make the database more feature-complete while preserving the structure and logic established in V2.0.

Deployment

Deployment Process

The purpose of this deployment process was to deploy a Flask-based web application onto the university's remote computer servers. The deployment required adapting the application, configuring the runtime environment, and resolving issues related to networking, server configuration, and application startup.

Preparing the Application for Deployment

The application was originally designed for local deployment. Several modifications were required before it could run properly on a remote server.

Removing the Development Startup behavior

Initially, the application performed several database operations every time the application started, such as:

- Deleting existing records
- Recreating default database entries
- Inserting test data

These operations were executed in run.py during application startup

This caused problems during deployment because Gunicorn creates multiple worker processes, which meant these operations ran multiple times, leading to duplicate database entries and repeated deletions.

To fix this, the database setup code was moved into a separate initialization script that is run manually once.

This ensured that the server startup process only creates the application instance and does not modify the database automatically.

Binding the Application to the Server Network Interface

Originally, the Flask server was configured as: app.run(host="localhost", port=5020)

This configuration restricts access to only the local machine.

Because the application needed to be accessed from other machines, the host was changed to:

host="0.0.0.0"

This allows the application to listen on all network interfaces of the server.

Using a Production Web Server

Flask's built-in development server is not designed for production use. It lacks features such as:

- Worker process management
- Proper request handling
- Scalability

For deployment, Gunicorn was used as the application server.

Gunicorn provides:

- Multiple worker processes
- Improved performance
- More reliable request handling

The application was started using (Bash):

Gunicorn -w 2 -b 0.0.0.0:5020 "run:app"

Where:

- "-w 2" specifies two worker processes
- "-b" specifies the network address and port

Running the Application

After configuration, the application was started on the server using:

- Gunicorn -w 2 -b 0.0.0.0:5020

The server reported:

- Listening at **http://0.0.0.0:5020**

This indicates the application is running and accepting connections.

Accessing the Application

Since the servers are behind the university firewall, there are two ways to access the application.

Option 1 - Internal Network Access

If connected to the campus network or VPN, the application can be accessed via the server's IP address:

- http://10.168.255.22:5020

- **Option 2 - SSH Port Forwarding**

A more reliable method is SSH tunneling.

From local machine:

- ssh -L 5020:127.0.0.1:5020 <username>@mcrugcomp03.ex.ac.uk

Then open in a browser:

- http://localhost:5020

This forwards the local port to the remote server.

Issues Encountered

When running Gunicorn with multiple workers, database initialization code executed multiple times. This caused:

- Duplicate records
- Repeated deletion of data
- Inconsistent application state

This occurred because Gunicorn worker imports the application module independently.

Resolution

Initialization logic was removed from application startup and moved to a separate script.

This also solved another issue ‘CRITICAL WORKER TIMEOUT’ due to it running multiple setup scripts at the same time.

Transparency

CO^2_e Research

Average Emissions by Vehicle Type (approximate, per km)

- Average Petrol Car: ~143–180 gCO2egCO sub 2 egCO2e/km.
- Average Diesel Car: ~170–173 gCO2egCO sub 2 egCO2e/km.
- New Cars (EU/UK 2023-2024): 102–107 gCO2egCO sub 2 egCO2e/km.
- Plug-in Hybrid (PHEV): ~28–163 gCO2egCO sub 2 egCO2e/km (highly dependent on electric vs. fuel usage).
- Electric Vehicle (EV): 0 gCO2e sub 2 egCO2e tailpipe emissions (roughly 50 gCO2e sub 2 egCO2e/km when accounting for average electricity grid mix

UK car CO_2 emissions by powertrain (tailpipe)

The most “official” UK-ready set for consistent comparisons is the UK Government GHG Conversion Factors (DESNZ). These factors include CO_2 + CH_4 + N_2O as CO_2e and are commonly used in reporting.

Average car (UK conversion factors, 2025) — per vehicle-km (tailpipe)

Car type	gCO_2e / km (tailpipe)	Notes
Diesel (average car)	173.0	tailpipe CO_2e
Petrol (average car)	162.7	tailpipe CO_2e
Hybrid (average car)	128.3	tailpipe CO_2e (non-plug-in hybrid)
Plug-in hybrid (PHEV, average car)	91.7	tailpipe CO_2e (depends heavily on charging/driving)
Battery electric (BEV)	0 (35-75)	tailpipe is zero (but not zero lifecycle)

Electric cars: add UK grid emissions (lifecycle context)

Tailpipe for BEVs is 0, but many reports also include lifecycle / electricity-generation emissions.

A UK-focused lifecycle estimate (example): ~46 gCO_2e/km for EVs in the UK in 2025 vs ~167 gCO_2e/km for a petrol car (lifecycle framing).

Average CO_2 emissions from buses and trains (UK)

Buses (per passenger-km, UK conversion factors)

From the UK Government conversion factors (Scope 3 "business travel – land" style factors, widely used as average intensities):

Mode	gCO_2e / passenger-km
Local London bus	68.8
Average local bus	103.9
Coach	27.8

These are per passenger-km, so occupancy matters a lot.

Trains (per passenger-km)

- Two very UK-useful options:
- UK Government conversion factor for National rail: 35.46 gCO_2e / passenger-km.
- Office of Rail and Road (ORR) published operational intensity for Great Britain rail: Latest year (Apr 2024–Mar 2025): 31 gCO_2e per passenger-km

Fuel burned during acceleration between speed limits (0–20, 0–30, 30–40, 30–60, etc.)

What solid research supports (directionally)

- Faster/more aggressive acceleration measurably increases fuel use (example finding: fuel consumption rose ~10.4% when acceleration rate increased from 1.0 to 4.0 mph/s).
- Aggressive driving can produce large real-world fuel penalties (example study reports ~23% increase vs normal driving in observed contexts).

The problem with “one universal table”

Fuel used for a speed jump depends heavily on:

- vehicle mass + drivetrain efficiency
- how hard you accelerate (engine operating point)
- gear changes, gradients, accessories, temperature
- hybrids/EVs: regen (what happens when you slow down again)

So, there isn’t one single UK “official” table that says “0–30 mph always burns X liters” for all cars. Could compute a physics-based baseline + apply a real-world uplift.

A practical, model-based set of estimates you can use

Below are per-acceleration-event estimates for a typical ~1500 kg car on level road.

Assumptions (transparent):

- petrol engine effective efficiency during accel: ~20%; diesel: ~25%
- real-world losses uplift: ×2 to ×5 (covers engine transients, rolling/aero during the accel, non-ideal operation)
- EV battery-to-wheels efficiency high; reported as Wh (energy drawn) with a smaller uplift

Estimated fuel/energy per single acceleration event:

Acceleration	Petrol (mL)	Diesel (mL)	EV energy (Wh)
0–20 mph	~18 to 44	~12 to 31	~24 to 39
0–30 mph	~39 to 99	~28 to 70	~53 to 88
20–30 mph	~22 to 55	~16 to 39	~29 to 49
30–40 mph	~31 to 77	~22 to 54	~41 to 69
40–60 mph	~88 to 219	~62 to 155	~118 to 196
30–60 mph	~118 to 296	~84 to 210	~159 to 265
0–60 mph	~158 to 394	~112 to 280	~212 to 353

How to interpret:

- This is the “cost of getting up to speed once”.
- If your driving pattern includes braking back down again, hybrids/EVs can recover some energy via regenerative braking, reducing net energy over a full accelerate-brake cycle.